

PROJECT REPORT

Complex Calculator

A menu-driven command-line calculator written in Python

PREPARED BY	Kanha
-------------	-------

PROGRAM	B.Tech, Aerospace Engineering
---------	-------------------------------

INSTITUTION	VIT Bhopal
-------------	------------

DATE	September 2026
------	----------------

Table of Contents

Abstract	4
1. Introduction	4
1.1 Background	4
1.2 Problem Statement	4
2. Objectives	4
3. Scope and Features	5
3.1 Features	5
3.2 Scope	5
4. System Requirements	5
5. System Design	5
5.1 Modular Architecture	5
5.2 Program Flow	6
5.3 Data Handling and Design Decisions	7
6. Implementation	8
6.1 Calculation Module (operations.py)	8
6.2 Input Module (input_handler.py)	8
6.3 Main Module (main.py)	8
6.4 Error Handling	9
7. Testing and Results	9
7.1 Test Method	9
7.2 Sample Runs	10
8. Limitations	12
9. Future Enhancements	12
10. Conclusion	12
References	12
Appendix A: Source Code	14

A.1 operations.py	14
A.2 input_handler.py	15
A.3 main.py	16

Abstract

Complex Calculator is a menu-driven, command-line calculator written in Python. Unlike a basic calculator that works on exactly two numbers, it lets the user enter any number of values one by one and then applies addition, subtraction, multiplication or division across the whole list. It also supports exponentiation, that is, raising a number to a power.

The program is organised into three modules: one for the calculations, one for displaying menus and reading user input, and one that controls the overall program flow. This keeps the mathematics, the user interaction and the control logic separate, which makes the code easier to read, test and extend. Common user mistakes, such as typing text instead of a number, entering no numbers at all, or dividing by zero, are reported with clear messages instead of stopping the program.

The program was verified with 15 scripted test cases covering normal use, invalid input and error conditions; all 15 produced the expected output. The report also records the current limitations of the program and suggests improvements for future work.

1. Introduction

1.1 Background

A calculator is one of the most common first programs because it brings together the core ideas of programming in a problem that is small and easy to verify: taking input, making decisions, repeating steps, using functions and handling errors. Python is well suited to this kind of project because of its readable syntax, its built-in numeric types and its simple input and output functions.

1.2 Problem Statement

Many simple calculator programs accept exactly two numbers and one operator, so a longer calculation such as $10 + 5.5 + 3$ has to be carried out in several separate steps. This project removes that restriction by letting the user supply a list of numbers of any length. It also aims to organise the program so that the calculation logic, the input handling and the program flow are clearly separated.

2. Objectives

The objectives of the project are:

- To build a menu-driven calculator that keeps running in the terminal until the user chooses to exit.
- To support addition, subtraction, multiplication and division on a list of any number of values.
- To support exponentiation (raising a number to a power).
- To validate user input and report common errors, such as non-numeric entries, an empty list of numbers and division by zero, without stopping the program.
- To divide the code into separate modules, each with a single responsibility.
- To verify the behaviour of the program with a set of test cases.

3. Scope and Features

3.1 Features

- Menu-driven interface that repeats until the user selects the exit option
- Addition, subtraction, multiplication and division of any number of values
- Exponentiation of a number by a given power
- Accepts integers and decimal numbers, including negative values
- Input validation with clear error messages
- Detection of division by zero
- Modular structure: calculations, input handling and program flow in separate files

3.2 Scope

The program is intentionally kept simple. It does not evaluate a whole expression typed on one line (such as $2 + 3 * 4$), it has no graphical interface, and it does not keep a history of earlier calculations. Each calculation uses a single operator applied to a list of numbers.

4. System Requirements

Table 1 lists what is needed to run the program.

Requirement	Details
Programming language	Python 3
Tested with	Python 3.12
External libraries	None. Only built-in Python features are used.
Operating system	Any system that runs Python (Windows, macOS, Linux)
Hardware	Any computer that can run Python; no special requirements
Interface	Terminal or command prompt

Table 1: System requirements

5. System Design

5.1 Modular Architecture

The program follows the principle of separation of concerns. Each of the three files has one clear responsibility, and only `main.py` depends on the other two (Figure 1). As a result, the calculation functions can be reused or tested without any keyboard input, and the menus and prompts can be changed without touching the mathematics.

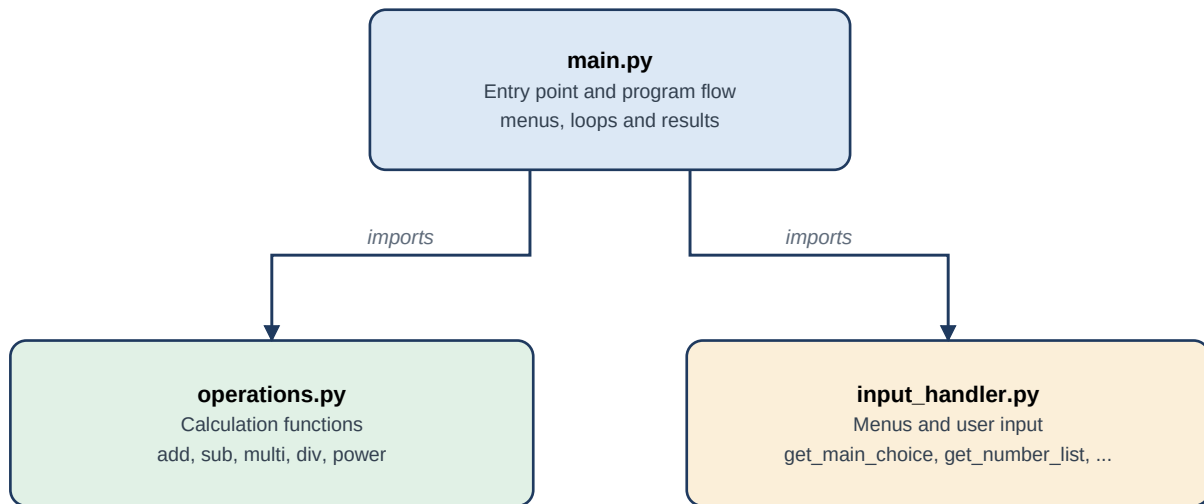


Figure 1: Modular architecture of the program

Module	Responsibility	Functions
operations.py	Arithmetic on lists of numbers; no input or output	add, sub, multi, div, power
input_handler.py	Displays the menus and reads and validates what the user types	get_main_choice, get_operation_choice, get_number_list, get_exponent_inputs
main.py	Entry point; controls the program flow and prints the results	main, run_basic_calculator, run_exponent

Table 2: Modules and their responsibilities

5.2 Program Flow

Figure 2 shows the overall flow. When `main.py` is run, the main menu is displayed and the user's choice is read. Choice 0 opens the basic calculator, choice 1 opens the exponent calculator and choice 2 prints a farewell message and stops the program. Any other input produces an error message and the menu is shown again.

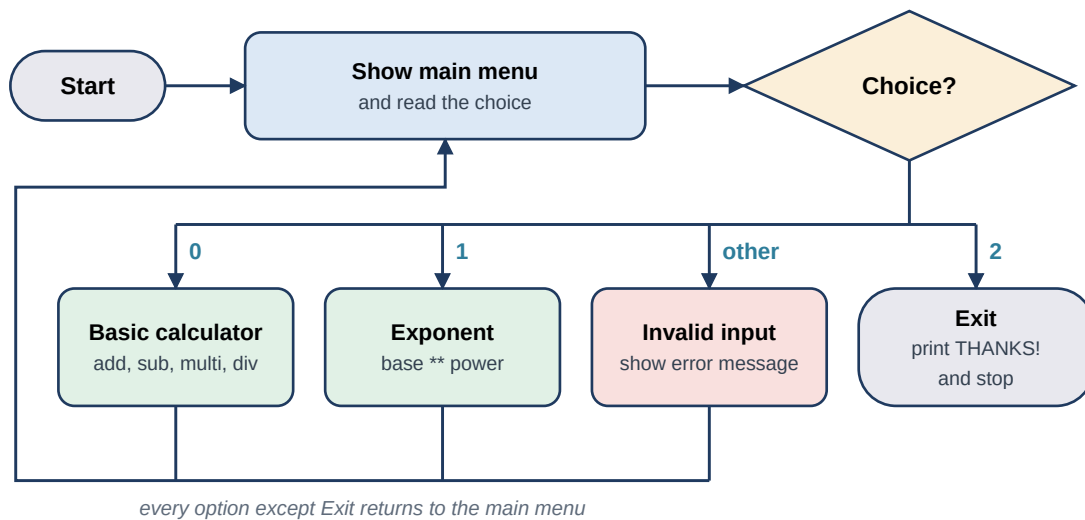


Figure 2: Overall program flow

The basic calculator (choice 0) works as follows:

1. The operator menu is shown and the user enters +, -, *, / or q.
2. If q is entered, the program prints "Thank You!" and returns to the main menu. Any other unknown symbol prints "invalid input" and the operator menu is shown again.
3. For a valid operator, the user enters numbers one by one. Each entry is converted to a floating-point number and added to a list; entries that are not numbers are rejected with an error message. Typing E or e ends the entry.
4. If the list is empty, an error message is shown and the operator menu appears again.
5. Otherwise the matching function from `operations.py` is called with the list, the result is printed, and the operator menu is shown again so that another calculation can be done.

5.3 Data Handling and Design Decisions

- Operands are stored in a Python list of floating-point numbers, so the same functions work for any number of values.
- Subtraction and division are applied from left to right. The first number is the starting value and each later number is subtracted or divided in turn. For example, 100, 5, 2 with division gives $100 / 5 / 2 = 10.0$.
- Every number is read as a float, so results are displayed with a decimal point (for example 15.0).
- The letter E (upper or lower case) marks the end of number entry, so the user does not have to say in advance how many numbers there will be.
- The main-menu choice is read as an integer. Anything that cannot be converted is treated as invalid and represented by -1.

6. Implementation

6.1 Calculation Module (operations.py)

This module contains only calculation logic and does no input or output. `add()` uses Python's built-in `sum()`. `sub()`, `multi()` and `div()` start from the first number (or from 1 for multiplication) and update a running result in a loop. `div()` checks every divisor before dividing and returns an error message if it finds a zero. `power()` returns `base ** exponent`. Listing 1 shows `div()`.

Listing 1: `div()` in `operations.py`

```
def div(number):
    if not number:
        return 0
    result = number[0]
    for num in number[1:]:
        if num == 0:
            return "Error: can't divide by 0."
        result /= num
    return result
```

6.2 Input Module (input_handler.py)

This module displays the menus and reads what the user types. `get_main_choice()` converts the menu choice to an integer and returns -1 if the conversion fails. `get_number_list()` keeps asking for numbers until the user types E or e. Each entry is converted with `float()` inside a try/except block, so a value that is not a number is rejected with an error message and the user is simply asked again (Listing 2).

Listing 2: `get_number_list()` in `input_handler.py`

```
def get_number_list():
    """Keep asking for numbers until the user types 'E' or 'e'."""
    number_list = []
    print("enter number one by one")
    print("type 'E' or 'e' to exit the calculator")

    while True:
        user_input = input("Enter a number(or 'E' to exit)")
        if user_input == 'E' or user_input == 'e':
            break
        try:
            num = float(user_input)
            number_list.append(num)
        except ValueError:
            print("Error: Invalid input")

    return number_list
```

6.3 Main Module (main.py)

`main.py` imports the functions from the other two modules and controls the program flow. `main()` is the outer loop: it shows the main menu and calls `run_basic_calculator()` or `run_exponent()`. Choosing the exit option prints a farewell message and leaves the loop with `break`. The line `if __name__ == "__main__":` at the end makes the program start only when the file is run directly (Listing 3).

Listing 3: `main()` in `main.py`

```
def main():
    while True:
        op = get_main_choice()

        if op == 0:
            run_basic_calculator()
        elif op == 1:
            run_exponent()
        elif op == 2:
            print("THANKS!")
            break # stop the program
        else:
            print("Invalid input, Please enter valid input")
```

6.4 Error Handling

Table 3 summarises how the program reacts to invalid input.

Situation	How it is handled	Message shown
Main-menu entry is not a number	<code>get_main_choice()</code> returns -1, which <code>main()</code> treats as invalid	Invalid input, Please enter valid input
Main-menu number other than 0, 1 or 2	Falls into the <code>else</code> branch of <code>main()</code>	Invalid input, Please enter valid input
Unknown operator in the basic calculator	Rejected; the operator menu is shown again	invalid input
Operand is not a number	<code>float()</code> raises <code>ValueError</code> , which is caught; number entry continues	Error: Invalid input
No numbers entered	The empty list is detected before calculating	Error: no number was entered.
Division by zero	<code>div()</code> checks each divisor	Error: can't divide by 0.
Exponent input is not a number	<code>ValueError</code> is caught in <code>run_exponent()</code>	Error: Please enter valid number

Table 3: Error handling in the program

7. Testing and Results

7.1 Test Method

The program was tested by running the real code with scripted keyboard input and comparing the text it printed with the expected output. Each of the 15 test cases in Table 4 covers one feature or one error condition. The input column lists what was typed, in order, separated by semicolons. For example, `0; +; 10, 5.5, E` means: main-menu choice 0, operator +, the numbers 10 and 5.5, then E to finish. Every session was closed by leaving the calculator with q and choosing the exit option 2, and a test counts as passed only if the expected text appeared and the program ended cleanly.

ID	Test description	Input	Expected output	Result
T1	Addition of decimal numbers	<code>0; +; 10, 5.5, E</code>	result= 15.5	Pass
T2	Subtraction (left to right)	<code>0; -; 10, 3, 2, E</code>	result= 5.0	Pass
T3	Multiplication, lowercase e to finish	<code>0; *; 2, 3, 4, e</code>	result= 24.0	Pass

ID	Test description	Input	Expected output	Result
T4	Division of several numbers	0; /; 100, 5, 2, E	result= 10.0	Pass
T5	Negative and decimal operands	0; *; -4, 2.5, E	result= -10.0	Pass
T6	Division by zero	0; /; 10, 0, E	result= Error: can't divide by 0.	Pass
T7	Non-numeric operand is rejected	0; +; 5, abc, 7, E	Error: Invalid input, then result= 12.0	Pass
T8	No numbers entered	0; +; E	Error: no number was entered.	Pass
T9	Unknown operator	0; x	invalid input	Pass
T10	Exponent	1; 2, 10	result: 1024.0	Pass
T11	Exponent with negative decimal power	1; 4, -0.5	result: 0.5	Pass
T12	Non-numeric exponent input	1; abc	Error: Please enter valid number	Pass
T13	Invalid main-menu number	9	Invalid input,Please enter valid input	Pass
T14	Non-numeric main-menu entry	abc	Invalid input,Please enter valid input	Pass
T15	Exit option ends the program	2	THANKS! and the program stops	Pass

Table 4: Test cases and results

All 15 test cases produced the expected output.

7.2 Sample Runs

The two runs below are real output from the program. Text typed by the user is shown in green, results in yellow and error messages in red.

Sample run 1: Addition, division and error handling

```
-----WELCOME-----
To add,subtract, multiply and divide--Enter: 0
For exponential-- Enter: 1
To exit-- Enter 2
enter your choice: 0
operations: add[+] subtract[-] multiply[*] divide[/] quit[q]
enter your choice: +
enter number one by one
type 'E' or 'e' to exit the calculator
Enter a number(or 'E' to exit)10
Enter a number(or 'E' to exit)abc
Error: Invalid input
Enter a number(or 'E' to exit)5.5
Enter a number(or 'E' to exit)E
result= 15.5
operations: add[+] subtract[-] multiply[*] divide[/] quit[q]
enter your choice: /
enter number one by one
type 'E' or 'e' to exit the calculator
Enter a number(or 'E' to exit)20
Enter a number(or 'E' to exit)0
Enter a number(or 'E' to exit)e
result= Error: can't divide by 0.
operations: add[+] subtract[-] multiply[*] divide[/] quit[q]
enter your choice: q
Thank You!
-----WELCOME-----
To add,subtract, multiply and divide--Enter: 0
For exponential-- Enter: 1
To exit-- Enter 2
enter your choice: 2
THANKS!
```

Sample run 2: Multiplication and the exponent option

```
-----WELCOME-----
To add,subtract, multiply and divide--Enter: 0
For exponential-- Enter: 1
To exit-- Enter 2
enter your choice: 0
operations: add[+] subtract[-] multiply[*] divide[/] quit[q]
enter your choice: *
enter number one by one
type 'E' or 'e' to exit the calculator
Enter a number(or 'E' to exit)4
Enter a number(or 'E' to exit)-2.5
Enter a number(or 'E' to exit)E
result= -10.0
operations: add[+] subtract[-] multiply[*] divide[/] quit[q]
enter your choice: q
Thank You!
-----WELCOME-----
To add,subtract, multiply and divide--Enter: 0
For exponential-- Enter: 1
To exit-- Enter 2
enter your choice: 1
enter the number: 9
enter the power: 0.5
result: 3.0
-----WELCOME-----
To add,subtract, multiply and divide--Enter: 0
For exponential-- Enter: 1
To exit-- Enter 2
enter your choice: 2
THANKS!
```

8. Limitations

The program meets its objectives, but it has the following known limitations.

- **Exponent edge cases.** The exponent option only guards against non-numeric input. Raising 0 to a negative power (for example `0 ** -1`) raises a `ZeroDivisionError`, and a result too large for a floating-point number (for example `10 ** 1000`) raises an `OverflowError`. Neither error is caught, so the program stops with an error message in these two cases.
- **Floating-point precision.** Numbers are stored as floating-point values, so some decimal results show tiny rounding differences. For example, `0.1 + 0.2` is displayed as `0.30000000000000004`.
- **Error message as a return value.** When a division by zero is found, `div()` returns a text message instead of a number. This is fine for displaying the result, but it would need to be an exception if the function were reused in other code.
- **One operator per calculation.** An expression that mixes operators, such as `2 + 3 * 4`, has to be calculated in separate steps.

9. Future Enhancements

- Catch `ZeroDivisionError` and `OverflowError` in the exponent option so that these inputs give a clear message instead of stopping the program.
- Raise exceptions in the calculation functions (for example, for division by zero) and handle them in `main.py`.
- Add more operations, such as modulus, square root, percentage and factorial.
- Keep a history of previous calculations.
- Automate the test cases with Python's `unittest` module.
- Build a graphical interface, for example with Tkinter.

10. Conclusion

Complex Calculator meets its objectives. It provides a simple menu-driven interface, performs the four basic operations on lists of numbers of any length, supports exponentiation and reports common input errors clearly. Splitting the program into calculation, input-handling and control modules made the code easier to read and to test, and all 15 test cases behaved as expected. The known limitations are small and well understood, and the suggested enhancements provide a clear path for extending the project.

References

- [1] Python Software Foundation. *The Python Tutorial*. Python 3 Documentation.
<https://docs.python.org/3/tutorial/>
- [2] Python Software Foundation. *Built-in Functions* (input, float, int, sum). Python 3 Documentation.
<https://docs.python.org/3/library/functions.html>

- [3] Python Software Foundation. *Errors and Exceptions*. Python 3 Documentation.
<https://docs.python.org/3/tutorial/errors.html>
- [4] van Rossum, G., Warsaw, B. and Coghlan, N. *PEP 8 - Style Guide for Python Code*.
<https://peps.python.org/pep-0008/>

Appendix A: Source Code

The complete source code of the three modules is listed below.

A.1 operations.py

```
1  # operations.py
2  # All the calculation functions of the calculator live here.
3
4
5  def add(number):
6      return sum(number)
7
8
9  def sub(number):
10     if not number:
11         return 0
12     result = number[0]
13     for num in number[1:]:
14         result -= num
15     return result
16
17
18  def multi(number):
19     if not number:
20         return 0
21     result = 1
22     for num in number:
23         result *= num
24     return result
25
26
27  def div(number):
28     if not number:
29         return 0
30     result = number[0]
31     for num in number[1:]:
32         if num == 0:
33             return "Error: can't divide by 0."
34         result /= num
35     return result
36
37
38  def power(base, exponent):
39     return base ** exponent
```

A.2 input_handler.py

```
1  # input_handler.py
2  # Everything that asks the user for input (menus, operator, numbers).
3
4
5  def get_main_choice():
6      """Show the main menu and return the choice as an int (-1 if invalid)."""
7      print("-----WELCOME-----")
8      print("To add,substract, multiply and divide--Enter: 0")
9      print("For exponential-- Enter: 1")
10     print("To exit-- Enter 2")
11
12     try:
13         return int(input("enter your choice: "))
14     except ValueError:
15         return -1
16
17
18  def get_operation_choice():
19     """Show the operations menu and return what the user typed."""
20     print("operations: add[+] substract[-] multiply[*] divide[/] quit[q]")
21     return input("enter your choice: ")
22
23
24  def get_number_list():
25     """Keep asking for numbers until the user types 'E' or 'e'."""
26     number_list = []
27     print("enter number one by one")
28     print("type 'E' or 'e' to exit the calculator")
29
30     while True:
31         user_input = input("Enter a number(or 'E' to exit)")
32         if user_input == 'E' or user_input == 'e':
33             break
34         try:
35             num = float(user_input)
36             number_list.append(num)
37         except ValueError:
38             print("Error: Invalid input")
39
40     return number_list
41
42
43  def get_exponent_inputs():
44     """Ask for the number and the power (raises ValueError if not numbers)."""
45     base = float(input("enter the number: "))
46     exponent = float(input("enter the power: "))
47     return base, exponent
```

A.3 main.py

```
1  # COMPLEX CALCULATOR
2  # Run this file to start the program:  python main.py
3
4  from operations import add, sub, multi, div, power
5  from input_handler import (
6      get_main_choice,
7      get_operation_choice,
8      get_number_list,
9      get_exponent_inputs,
10 )
11
12
13 def run_basic_calculator():
14     """Add / subtract / multiply / divide a list of numbers."""
15     while True:
16         choice = get_operation_choice()
17
18         if choice == 'q':
19             print("Thank You!")
20             break
21         if choice not in ['+', '-', '/', '*', 'q']:
22             print("invalid input")
23             continue
24
25         number_list = get_number_list()
26         if not number_list:
27             print("Error: no number was entered.")
28             continue
29
30         if choice == '+':
31             print("result=", add(number_list))
32         elif choice == '-':
33             print("result=", sub(number_list))
34         elif choice == '*':
35             print("result=", multi(number_list))
36         elif choice == '/':
37             print("result=", div(number_list))
38
39
40 def run_exponent():
41     """Raise a number to a power."""
42     try:
43         base, exponent = get_exponent_inputs()
44         print("result:", power(base, exponent))
45     except ValueError:
46         print("Error: Please enter valid number")
47
48
49 def main():
50     while True:
51         op = get_main_choice()
52
53         if op == 0:
54             run_basic_calculator()
55         elif op == 1:
56             run_exponent()
57         elif op == 2:
58             print("THANKS!")
59             break # stop the program
60         else:
61             print("Invalid input, Please enter valid input")
62
63
```

```
64 if __name__ == "__main__":  
65     main()
```